An Axiomatic Concurrency Model

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

S A +

S A

R A +

R

A

SB+

S B –

C A +

C A

R B +

R B _

C B +

C B _

po

po

r

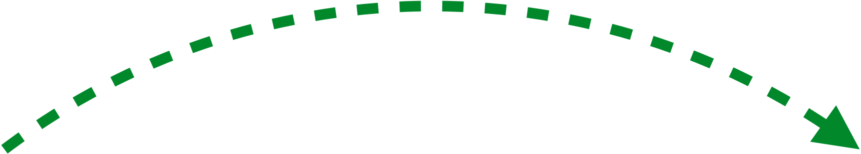
r

po

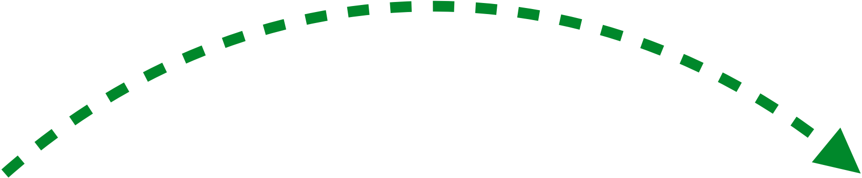
po

po

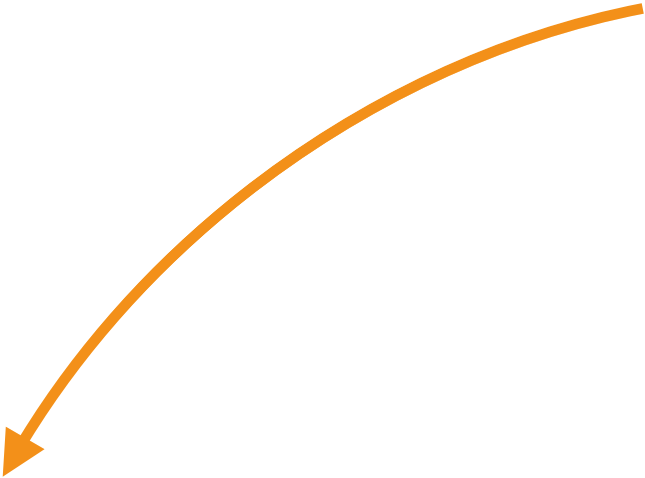
po



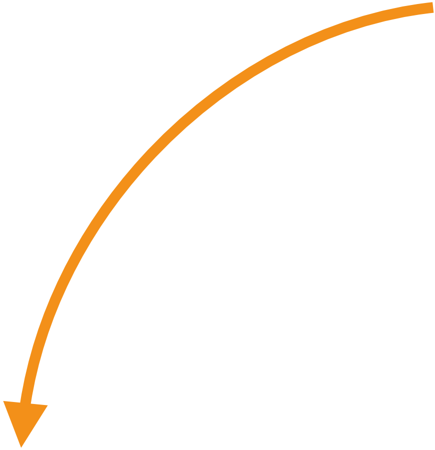
r



r



COsrc



COlog

We derive the run relation from behaviour spawn to run

```
update(src, +100)
```

```
update(src, -100)
```

Candidate executions are tuples of $(\mathbb{E}, \mathrm{po}, \mathrm{co})$

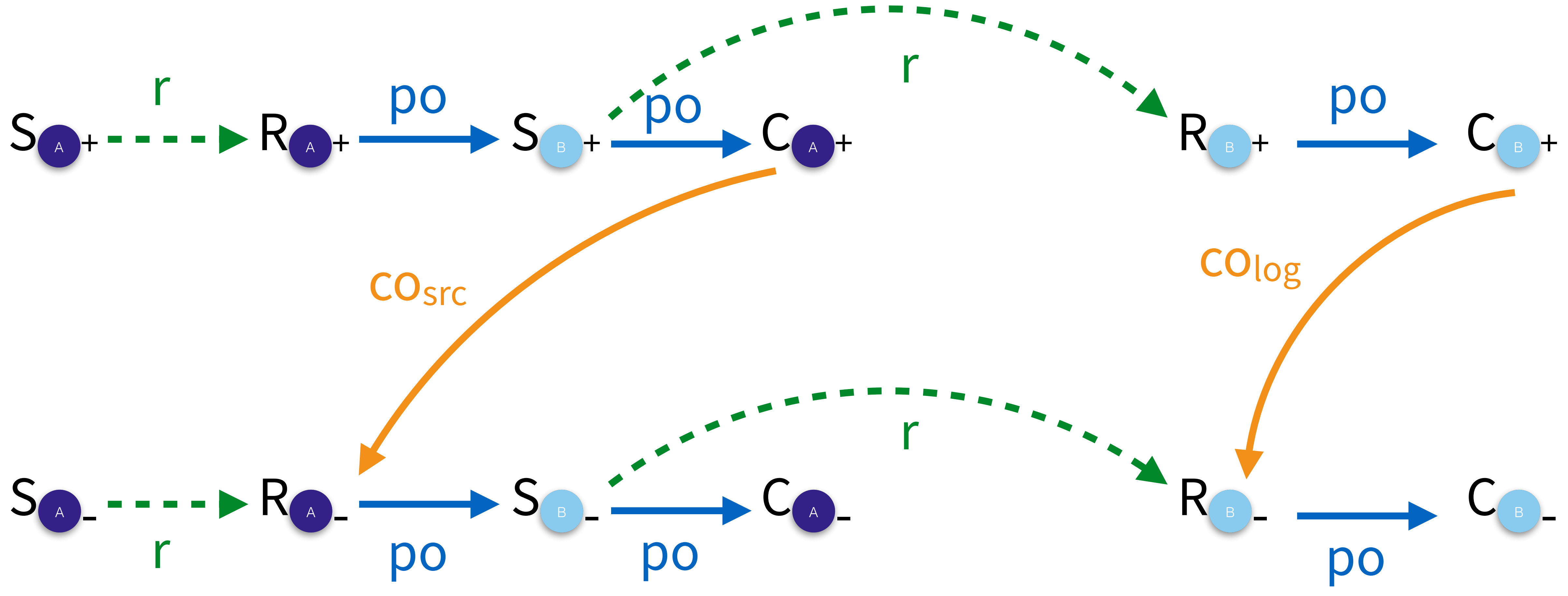
An Axiomatic Concurrency Model

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

```
update(src, +100)
```

```
update(src, -100)
```

Candidate executions are tuples of ($\mathbb{E}$, po, co)



We derive the run relation from behaviour spawn to run

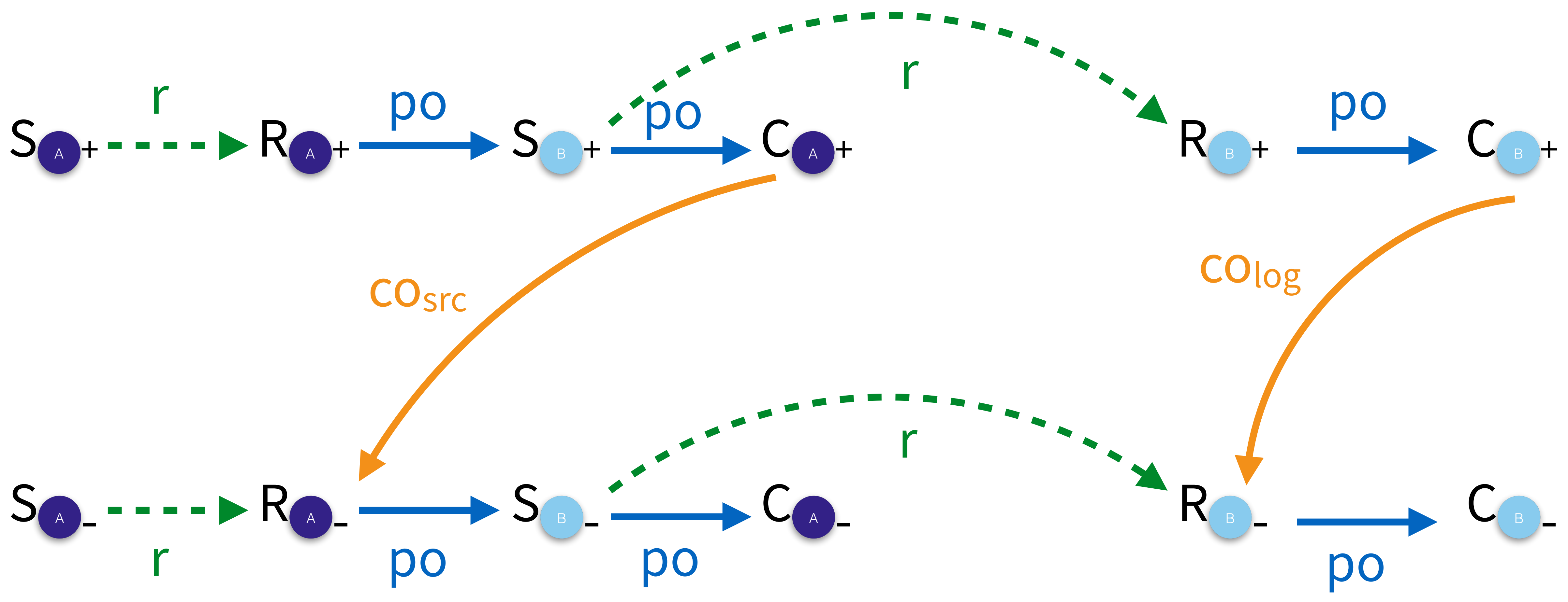
An Axiomatic Concurrency Model

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

`update(src, +100)`

`update(src, -100)`

Candidate executions are tuples of $(\mathbb{E}, \text{po}, \text{co})$



We derive happens before

$$\text{hb} = \{(C_i, R_j) \mid S_i\ (\text{po} \cup \text{r} \cup \text{co})^{+}\ S_j \wedge \text{cowns}(i) \cap \text{cowns}(j) \neq \emptyset \ldots\}$$

Is there a causal path between the spawns of two behaviours that require the same cowns?